Fase 4 - Aplicacion Flask y Nginx Reverse Proxy

Objetivo

Desarrollar una aplicacion web propia en Python utilizando Flask, contenerizarla con Docker, desplegarla mediante Docker Compose y exponerla a traves de Nginx como Reverse Proxy.

Arquitectura inicial

El laboratorio partio de una arquitectura simple donde la aplicacion Flask era accedida directamente desde el navegador mediante el puerto 5000.

Desarrollo de la aplicacion

Se creo una aplicacion Flask sencilla utilizando app.py. La aplicacion mostraba un mensaje de validacion y el hostname del contenedor para demostrar que la respuesta provenia de Docker.

Dependencias y Dockerfile

Se genero un archivo requirements.txt con Flask como dependencia y un Dockerfile propio utilizando una imagen base Python. Esto permitio construir una imagen personalizada en lugar de utilizar imagenes preconstruidas de terceros.

Docker Compose para Flask

Se decidio utilizar Docker Compose para desplegar la aplicacion. Se analizo la diferencia entre docker run y docker compose, entendiendo que Compose permite describir infraestructura como codigo y versionarla en Git.

Validacion de la aplicacion

La aplicacion fue desplegada exitosamente y accesible desde Windows mediante el puerto 5000. Se verifico el funcionamiento observando el contenido generado por Flask y el hostname del contenedor.

Intento con Nginx Proxy Manager

Se realizo una prueba utilizando Nginx Proxy Manager. Durante la configuracion se identificaron limitaciones relacionadas con dominios inexistentes y certificados Lets Encrypt. Se concluyo que era preferible comprender primero Nginx tradicional antes de incorporar herramientas de abstraccion.

Implementacion de Reverse Proxy

Se creo una configuracion personalizada de Nginx utilizando proxy_pass para reenviar el trafico recibido hacia el contenedor Flask. Se aprendio el concepto de Reverse Proxy y su uso habitual en ambientes productivos.

Networking Docker

Se aprendio que Docker Compose crea una red interna automaticamente. Gracias a ello Nginx pudo comunicarse con Flask utilizando el nombre del servicio flask-app sin necesidad de direcciones IP fijas.

Problemas encontrados

Se produjo un error en el montaje de volumenes debido al uso incorrecto de rutas relativas y absolutas. Tambien se detecto un conflicto entre directorios y archivos al intentar montar default.conf. Adicionalmente Nginx continuaba mostrando su pagina por defecto porque no estaba cargando la configuracion esperada.

Resolucion de problemas

Se corrigieron las rutas de montaje, se recrearon los contenedores y se verifico la configuracion efectiva de Nginx. Finalmente el trafico pudo ser reenviado correctamente desde Nginx hacia Flask.

Resultado final

La aplicacion Flask dejo de estar expuesta directamente al usuario. Todo el trafico paso a atravesar Nginx, quedando la arquitectura Browser -> Nginx -> Flask funcionando correctamente mediante Docker Compose.

Competencias adquiridas

Flask, Python Web Apps, Dockerfile, imagenes personalizadas, Docker Compose, Reverse Proxy, Nginx, proxy_pass, redes Docker, DNS interno de Docker, volumenes, troubleshooting de contenedores, troubleshooting de Nginx e Infrastructure as Code.

Estado del laboratorio

Al finalizar la fase el laboratorio cuenta con Linux endurecido, GitHub, Docker, Docker Compose, una aplicacion propia en Flask y un Reverse Proxy Nginx operando sobre una arquitectura multicontenedor.